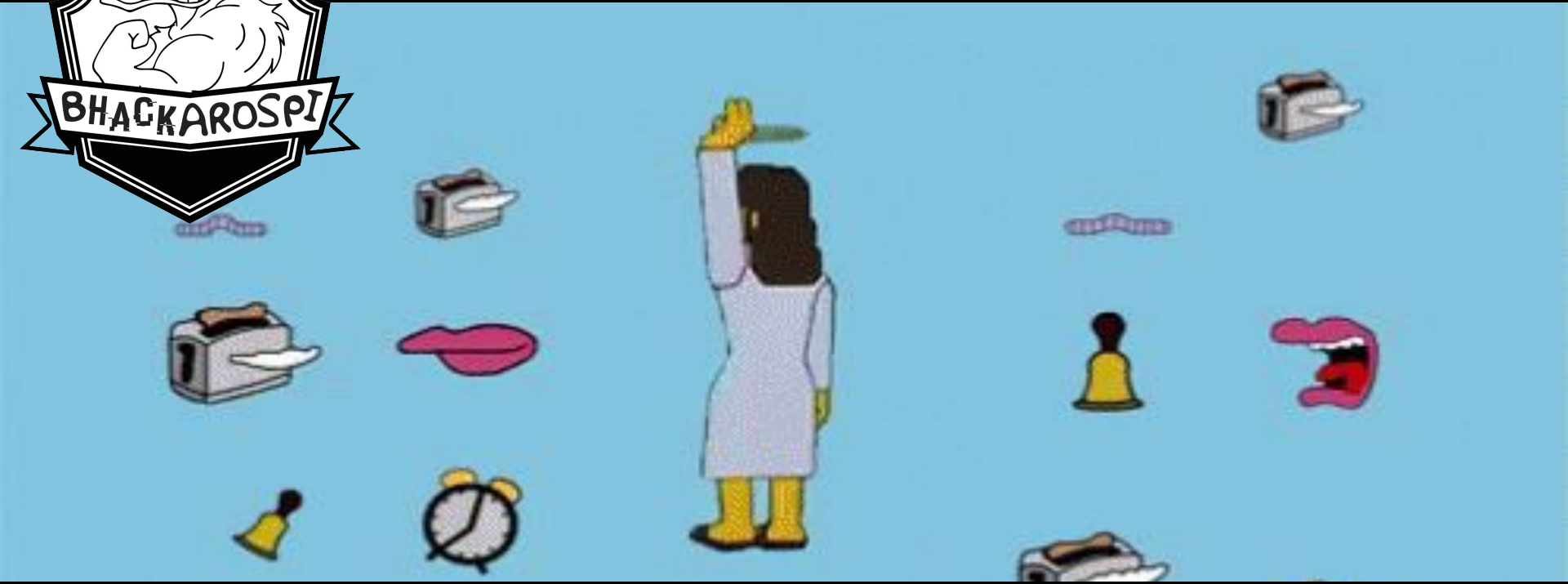
The BHACKAROSPI presents

ROMHACK 2026 CAMP



# Stupid Sexy Bearer Tokens

Deep dive into exploding OIDC sessions and hardening with DPoP

# DISCLAIMER

This deck may contain a higher than average number of The Simpsons memes.



# Hello, I’m Troy McClure...

Hello, I'm Troy McClure.

You kids might remember me from such cybersecurity documentaries as

*“Deepfake My Wife”*:
***The AI nightmare that ruined thanksgiving***

and

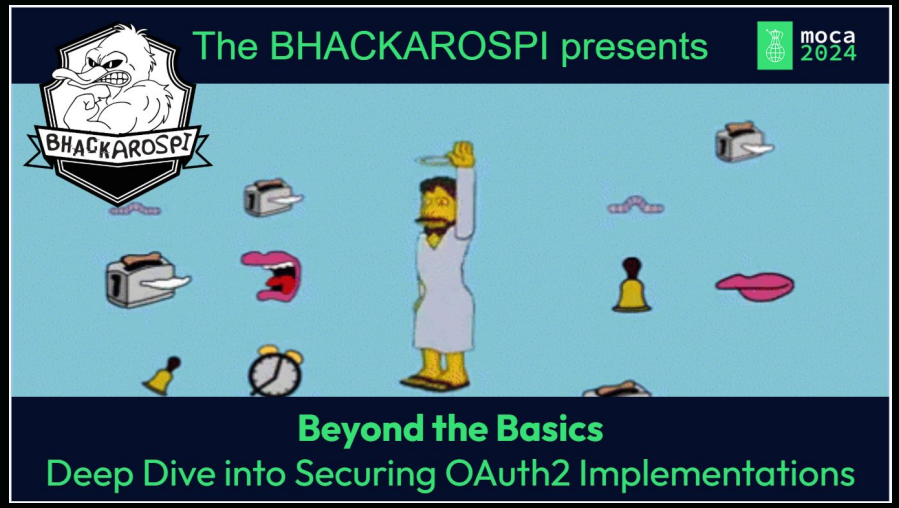
*“Don’t eat those cookies”*:
***Session management in HTTP***

or perhaps

*“Flamethrowers and Unit tests”*:
***How to live bug-free***



# If you wanna catch-up on the basics




moca 2024

https://youtu.be/ehSkbR-YuZw

# OAuth 2.0 General Overview



User




User Browser
Or mobile application

Request Token

Token



Identity Provider

Get resource
with Token

Resource



Resource Server



# Ok.. but what's a token?



Client



Res. Serv.

```
POST /res HTTP/1.1
Host: jetmarket.bhackarospi.it
Authorization: Bearer b64(header).
```

**Header**

`typ` : MUST be set to `"JWT"`

`alg` : The signature algorithm



# Ok.. but what's a token?



Client



Res. Serv.

```
POST /res HTTP/1.1
Host: jetmarket.bhackarospi.it
Authorization: Bearer b64(header).b64(payload).
```

## Header

`typ` : MUST be set to `"JWT"`

`alg` : The signature algorithm

## Payload

`iss` : Who issued the token

`sub` : Whom the token represents

`aud` : Who the token is intended for

`exp` : When the token expires

`nbf` : When the token becomes active

`iat` : When the token was created

`azp` : Who requested the token

`scope` : What the user has consented to

# Ok.. but what's a token?



Client



Res. Serv.

```
POST /res HTTP/1.1
Host: jetmarket.bhackarospi.it
Authorization: Bearer b64(header).b64(payload).sign(b64(header),b64(payload))
```

## Header

`typ` : MUST be set to `"JWT"`

`alg` : The signature algorithm

## Payload

`iss` : Who issued the token

`sub` : Whom the token represents

`aud` : Who the token is intended for

`exp` : When the token expires

`nbf` : When the token becomes active

`iat` : When the token was created

`azp` : Who requested the token

`scope` : What the user has consented to

# Today, we're talking about Sexy Tokens

## Stateless and Scalable

The token is self-contained, the backend doesn't need to look it up or validate each token with the central authentication server.

## Cross-domain and Cross-platform

Each request is tagged with a token, the recipient doesn't really care where the request comes from. If the token is valid, the request is legitimate.

## Built-in security features

Native claims allow to specify strict requirements on token validity, while allowing fine-grained control over what the user has granted.






DEMO TIME!

# What happened? Tokens are...

## Immutable

Once issued, tokens are just JSON and valid tokens cannot be “edited” if the user decides to logout or invalidate it.

## Easily misconfigured

There are more than 30 RFCs on OAuth. Security relies heavily on configuring properly both Authorization Server and Resource Server.

## Possession means ownership

By definition, a bearer token grants access to whichever entity holds it. No questions asked. By default, the Resource Server must check the validity of the token, not *who* is presenting it.



# Possession means ownership?



If the JWT is used as Bearer token, yes.

**How to bind it to the original requester?**
We need to introduce sender-constraining or proving proof of possession (PoP)

In OAuth 2.0, RFC 7800 introduces the architecture for PoP

- A token may contain a confirmation claim `cnf` that indicates a key to be used to verify requests.
- The client must sign or encrypt a portion of the request using a key to prove they "possess" the secret bound to the token.

# OAuth 2.0 Demonstrating Proof of Possession (DPoP)

Add **sender-constraints** to access and refresh tokens, effectively binding the token to a specific cryptographic key pair held by the client.

### Header

- `typ` : MUST be explicitly set to `"dpop+jwt"`
- `alg` : The asymmetric signature algorithm
- `jwk` : The public key corresponding to the private key used to sign the proof

### Payload

- `htm` : The HTTP method of the request
- `htu` : The target HTTP URI (without query or fragment parts)
- `iat` : A creation timestamp
- `jti` : A unique identifier to prevent replay attacks
- `ath` : A hash of the access token being presented
- `nonce` : An *optional* server-provided value to prevent pre-generated proofs



# Deep dive! OAuth 2.0 + DPoP



Client



AuthZ Serv.

```
POST /token HTTP/1.1
Host: keycloak.bhackarospi.it
Content-Type: application/x-www-form-urlencoded
DPoP: eyJ0eXAiOiJkcG9wK2p3dCI...
grant_type=authorization_code
```

Header

```
"typ": "dpop+jwt",
"alg": "ES256",
"jwk": {...}
```

Payload

```
"jti": "-BwC3ESc6acc2lTc",
"htm": "POST",
"htu": "https://keycloak.bhackarospi.it/token",
"iat": 1562262616
```

# Deep dive! OAuth 2.0 + DPoP



Client



AuthZ Serv.

```
POST /token HTTP/1.1
Host: keycloak.bhackarospi.it
Content-Type: application/x-www-form-urlencoded
DPoP: eyJ0eXAiOiJkcG9wK2p3dCI...
grant_type=authorization_code
```

Provide DPoP-bound Access Token

```
"sub": "homer@bhackarospi.it",
"iss": "https://keycloak.bhackarospi.it",
"nbf": 1562262611,
"exp": 1562266216,
"cnf": {"jkt": "0ZcOCORZNYy-DWpqq30jZyJG…"}
```

# Deep dive! OAuth 2.0 + DPoP



Client



Res. Serv.

```
POST /res HTTP/1.1
Host: jetmarket.bhackarospi.it
Content-Type: application/json
Authorization: DPoP Kz~8mXK1EalYznwH-LC-1fBAo.4Ljp~zsPE_NeO.gxU...
DPoP: eyJ0eXAiOiJkcG9wK2p3dCIsImFsZy...
```

Header

```
"typ": "dpop+jwt",
"alg": "ES256",
"jwk": {...}
```

Payload

```
"jti": "c2a8b-98fc-45",
"htm": "POST",
"htu": "https://jetmarket.bhackarospi.it/res",
"iat": 1562262618,
"ath": "fUHyO2r2Z3DZ53EsNrWBb0xWX..."
```

# Deep dive! OAuth 2.0 + DPoP



Client



Res. Serv.

**Option 1:**
Error + Nonce

```
POST /res HTTP/1.1
Host: jetmarket.bhackarospi.it
Content-Type: application/json
Authorization: DPoP Kz~8mXK1EalYznwH-LC-1fBAo.4Ljp~zsPE_NeO.gxU...
DPoP: eyJ0eXAiOiJkcG9wK2p3dCIsImFsZy...
```

```
HTTP/1.1 401 Unauthorized
WWW-Authenticate: DPoP error="use_dpop_nonce", error_description="Resource
server requires nonce in DPoP proof"
DPoP-Nonce: eyJ7S_zG.eyJH0-Z.HX4w-7v
```



# Deep dive! OAuth 2.0 + DPoP



Client



Res. Serv.

**Option 2:**
OK +
Nonce

```
POST /res HTTP/1.1
Host: jetmarket.bhackarospi.it
Content-Type: application/json
Authorization: DPoP Kz~8mXK1EalYznwH-LC-1fBAo.4Ljp~zsPE_NeO.gxU...
DPoP: eyJ0eXAiOiJkcG9wK2p3dCIsImFsZy...
```

```
HTTP/1.1 200 OK
Content-Type: application/json
DPoP-Nonce: eyJ7S_zG.eyJbYu3.xQmBj-1
```


DEMO TIME!

# What happened (Part I)?

## How DPoP Prevented the attack?

### The binding (@ issuance)

- The AS embeds the SHA-256 thumbprint of the client's public key into the token `cnf.jkt`
- Token type is `DPoP`, not Bearer

### Proof of possession (@ every request)

- The client attaches a fresh signed proof (`DPoP` header), signed with the private key
- Proof carries: `jti` (JWT ID), `htm` (HTTP method), `htu` (HTTP target URI), `iat` (proof creation timestamp), `ath` (access token hash)

### The check (@ resource server)

- Recompute the proof key's thumbprint, must equal the token's `cnf.jkt`
- Verify: `jti`, `htm`, `htu`, `iat`, `ath`



# Isn’t PKCE the same thing though?

## PKCE

1. Client generates a random `code_verifier`
2. Computes `code_challenge = base64url(SHA-256(code_verifier))`
3. Sends `code_challenge` in the authorization request
4. Sends `code_verifier` in the token request
5. AS re-hashes the verifier and compares to the stored challenge

| | PKCE | DPoP |
|---|---|---|
| **Protects** | the authorization `code` exchange | the `token` in use |
| **When** | during login ( `code` → `token` ) | after login (every API call) |
| **Against** | code interception / injection | token theft & replay |
| **Lifetime** | one-time, at token request | continuous, one proof per request |
| **Binds to** | a one-time secret | a long-lived key pair |



# DPoP Is Only As Strong As Its Config

Common gaps:

- DPoP not enforced → attackers just omits the proof → gets a plain Bearer
- Extractable private key → attackers just extract the key
- No downgrade protection on the RS → a DPoP-bound token is accepted as Bearer
- No nonce → proofs can be pre-generated and replayed offline
- Weak proof validation → forgery & replay:
  - accepting `alg: none`
  - not checking `jti`, `htm`, `htu`, `iat`, `ath`

OWASP®

OWASP Top 10:2025
A02:2025 Security Misconfiguration




Demo scenarios: stealing DPoP exportable private key

# What happened (Part II)?

**The attackers stole the Private Key!**

## HOW the PK was stolen

- Extractable key → Malicious code can reads it, just like the token
  - stored as `extractable: true` or as a `raw JWK` in browser storage

## WHY the attackers can then use the stolen PK

- With the PK, attackers recompute the same JWK thumbprint → it matches the token's `cnf.jkt`
- They sign a valid proof for any `htu` / `htm`, with a fresh `jti` / `iat` and the correct `ath`
- The RS sees a perfectly valid DPoP request → RS serves the data



# … how to fix it? Protect the PK!

1. Make the key impossible to exfiltrate
   a. Set `extractable: false`: sign with it, never read it
   b. Store `CryptoKey` object in `IndexedDB`, never a raw JWK string
   c. Hardware-backed where available

2. Limit the blast radius
   a. Short-lived access tokens; bind the refresh token to the same key
   b. Consider a BFF (keep tokens out of the browser entirely)



# … how to fix it? Protect the RS from Downgrade!

1. The rule
   a. A DPoP-bound token must never be accepted as `Bearer`
   b. RS logic: if the token carries `cnf.jkt` → require `Authorization: DPoP` + a valid proof; reject `Bearer`

2. Signal it correctly
   a. On challenge: `WWW-Authenticate: DPoP` (advertise accepted `algs`)





# … how to fix it? Protect the RS with Nonce + JTI!

1. Server nonce
   a. Server issues an unpredictable value; the proof must echo it in the `nonce` claim
   b. Defeats pre-generation and offline replay



2. JTI replay cache
   a. Store seen `jti` for the proof's short validity window → reject duplicates

3. Tight freshness
   a. Accept `iat` only within a short window (seconds / minutes)




Demo scenarios: hardened DPoP and Resource Server

# What happened?

**Why the fuck did it happen again?**

**Despite all my hardening?!**

I implemented DPoP.

I hardened the configuration.

I protected the private key.

**And yet… the attacker can still use the stolen token?!**



# What happened?

***The attacker doesn't need your key***
***if your browser can use it for them!***

**DPoP ≠ XSS Protection***

**XSS can generate valid DPoP requests - without ever exporting the private key!**

Attacker simply **uses the legitimate signing capability** available in the **compromised browser context**

Similar to CSRF bypasses through XSS

**XSS is out of scope for this talk - fix your own XSS!*



# Why are we making our lives harder?

***Stolen token shouldn't mean unlimited access***

**Without DPoP**

Token stolen → replay it anywhere → use it whenever needed

**With DPoP**

**Bind token** usage to the client (Chrome DBSC, is that you?)

Force the attacker to **operate through the victim's browser**

**Limit** token **usability** and **shrink the attack window**



# Annoying the Attacker

We’re NOT solving completely the problem

We’re NOT stopping the attacker

We’re **making their job harder, slower, and more constrained**

We want the attacker **to give up before compromising the system**



# So DPoP is great for...

**Neutralizing stolen access tokens**
Stolen tokens are useless without the client's private key

**Preventing access token replay**
Binds requests to specific HTTP methods, URIs, and tokens.

**Providing application-layer sender constraints**
Works directly in browsers and SPAs where mTLS is not doable.

**Protecting refresh tokens**
Stolen public client refresh tokens cannot be used elsewhere.

**Breaking CORS**
If no `Access-Control-Expose-Headers` and `Access-Control-Allow-Headers` are used when forcing nonce usage.



# ... but it still fails when

**Attacker has user-context access**
Attackers can forge signatures through the browser during active sessions, for example with XSS

**Attack targets request body integrity**
DPoP signs method and URI, completely ignores the request body. DPoP requires HTTPS and does not replace transport security.

**Fundamental security hygiene is omitted**
Skip server nonces, short token lifetimes, or strict CSP leaves room for proof pre-generation and replay attacks

**Developers do not follow all guidelines**
Vulns. like nonce downgrade, *pre-signed proof replay*, *JWT swapping*, or *key substitution* is going to bypass verification if implementation is haphazard



# Is that all?






# BHACKAROSPI

**We did our best and we failed miserably.**

*The lesson is never try...*

## Executive Nerds

| | | |
|---|---|---|
| Jiraky | as | Troy McClure |
| TocTou | as | SideShowBob |
| Reymerk | as | the scientician |
| Snorlax | as | Sebastian Cobb |
| Grasp4 | as | Professor Frink |

ROMHACK 2026 CAMP